We've maintained the same structure of the code, there's one ractor that is reading the file and another ractor that is actually doing the word count.

Here's more or less how this plays out, with the understanding that because this is parallel code, the exact order may differ slightly.

First, we create the reader ractor. The block starts immediately (or is scheduled to start immediately), and opens the file and scans the first line, at which point it blocks on Ractor.yield with the first word scanned.

Moving down the file, the counter ractor is created, with—and this is important—the reader as an argument. The counter block is now executed with the reader passed in as source. We have to pass the ractor to the block because the ractor is isolated. The block inside the ractor does not have access to the local variable reader.

Inside the counter ractor block, we build an empty hash, then a while loop on source.take. Each time we call source.take, we grab the most recent value yielded by the reader, and the reader continues forward until it blocks on the next yield call.

Eventually, the reader runs out of words in the file and hits the nil at the end of the block. Subsequently, the last source.take returns nil and ends the while loop and the counter returns the result.

After both blocks is the counter.take call, which will block the main ractor until the counter ractor is ready to return a value. Since the counter ractor doesn't yield anywhere, that call waits until it exits, and then the final value is available to take. It's a good thing we have that take call blocking, because if we didn't block on something in the main ractor, the code would terminate and all the internal ractors would be stopped.

Having pulled that last value, we then print out the results.

And it works. The two ractors run in parallel. But I don't really like that the reader is blocked on every word, it seems like you'd rather allow the reader to get as far ahead as it can, and let the counter catch up.

You can do that by reversing the direction of the interaction, like this:

tut_threads/ractor_word_count_flipped.rb

```
counter = Ractor.new(name: "counter") do
  result = Hash.new(0)
  while (word = Ractor.receive)
    result[word] += 1
  end
  result
end

Ractor.new(counter, name: "reader") do |worker|
  File.foreach("./testfile") do |line|
    line.scan(/\w+/) do |word|
      worker.send(word.downcase)
    end
  end
  worker.send(nil)
end

counts = counter.take
counts.keys.sort.each { |k| print "#{k}:#{counts[k]} " }
```